

PRÁCTICA · JAVASCRIPT EN EL NAVEGADOR

Una calculadora, tres caminos

Buenas, soy Ramón Abramo, profesor con más de 25 años de experiencia docente. Hoy os voy a explicar tres formas distintas de programar la misma calculadora. Mismo HTML, mismo CSS, mismo resultado en pantalla… y tres maneras de pensar el código.

Solución 0 · La de clase | Solución 1 · Un escuchador por botón | Solución 2 · Delegación de eventos

Ramón Abramo · Explicación de las tres soluciones

MAPA DE LA SESIÓN



¿Qué vamos a hacer hoy?

Este es el recorrido que vamos a hacer, en este orden.

Lo común

El HTML que comparten las tres, la «memoria» de la calculadora y los 7 hitos.

0 · La de clase

Tres grupos de botones y una función `calcular()` que no toca nada de fuera.

1 · Un escuchador por botón

Cada botón avisa a una función con nombre propio. Muy ordenadito.

2 · Delegación

Un solo escuchador en el teclado entero gracias al burbujeo.

Comparativa

Las tres frente a frente, trampas típicas y cuándo usar cada una.

Fuentes

Documentación oficial para ampliar en casa.

LO COMÚN · EL HTML



Los botones llevan una etiqueta pegada

```
<output id="display" class="display">0</output>

<div class="teclado">
  <button data-accion="borrar">C</button>
  <button data-operador="/">/</button>
  <button data-numero="7">7</button>
  <button data-numero="8">8</button>
   ...
  <button data-operador="+">+</button>
  <button data-numero="0">0</button>
  <button data-accion="igual">=</button>
</div>
```

Fijaos en los atributos que empiezan por `data-`. Son atributos inventados por nosotros, y HTML los permite precisamente para eso: guardar información extra en una etiqueta.

- `data-numero` → los dígitos del 0 al 9.
- `data-operador` → `+`, `-`, `*`, `/`.
- `data-accion` → `igual` o `borrar`.



Para entenderlo: son como las etiquetas de los botes de la cocina. El bote es igual por fuera, pero la etiqueta te dice si dentro hay sal o azúcar.

Desde JavaScript los buscamos con `querySelectorAll('[data-numero]')`[1] y leemos su valor con `boton.dataset.numero`[2]. Ojo: `data-numero` en HTML se convierte en `dataset.numero` en JS.

LO COMÚN · LA MEMORIA



¿Qué tiene que recordar una calculadora?

Haced la prueba mental: tecleáis `15`, pulsáis `+`, tecleáis `4` y pulsáis `=`. ¿Qué tiene que tener apuntado la calculadora para poder decir `19`?

Cuatro cosas, ni una más

1. **Lo que hay en pantalla** ahora mismo (`4`).
2. **El número de antes**, el que estaba cuando pulsasteis el operador (`15`).
3. **La operación pendiente** (`+`).
4. **Una bandera**: «el próximo dígito empieza un número nuevo, no lo pegues al anterior».



Para entenderlo: pensad en un camarero apuntando en una libreta. Tiene que anotar el primer plato, qué queréis hacer con él y el segundo plato. Si no apunta, se le olvida todo.

Esa bandera es la que más lío da. Sin ella, al teclear `15`, `+` y `4` veríais `154` en pantalla. Y no, eso no es una suma.

Cada solución llama a estas variables de forma distinta, pero las cuatro están siempre ahí.

LO COMÚN · LAS VARIABLES DE CADA SOLUCIÓN



Misma memoria, distintos nombres

QUÉ GUARDA	SOLUCIÓN 0	SOLUCIONES 1 Y 2
Lo que hay en pantalla	Se lee directamente de `pantalla.textContent` [3]	`operandoActual` (texto, empieza en `'0'`)
El número de antes	`numeroAnterior`	`operandoAnterior`
La operación pendiente	`operadorPendiente`	`operacion`
La bandera de «número nuevo»	`empezarNumero`	`reiniciarDisplay`
Extra	`numeroActual` (copia en número de lo que hay en pantalla)	—



Ojo: la Solución 0 usa la pantalla como «libreta» del número actual. Las Soluciones 1 y 2 lo guardan en una variable y luego lo pintan en pantalla. Esa diferencia parece pequeña, pero cambia cómo se escribe todo lo demás.

LO COMÚN · LOS 7 HITOS



Se construye por hitos, no de golpe

#	HITO	PRUEBA QUE TIENE QUE SALIR BIEN
1	Escribir números (sin «07»)	`1 2 3` → `123`
2	Guardar en memoria al pulsar `+`	`15 + 4` → se ve `4`
3	Calcular con `=`	`7 + 8 =` → `15`
4	Resta, multiplicación y división con `switch`	`9 - 4 = 5` · `6 * 3 = 18` · `20 / 4 = 5`
5	Operaciones encadenadas	`5 + 3 +` → ya se ve `8`
6	Borrar con `C`	`999 C` → `0`
7	Errores y decimales	`8 / 0 =` → `Error`

Mi recomendación es que no paséis al hito siguiente hasta que el anterior funcione en pantalla. Programar diez cosas y probarlas todas al final es la receta perfecta para no saber dónde está el fallo.

LO COMÚN · HERRAMIENTAS QUE VAIS A VER



Tres piezas de JavaScript que salen en todas

`querySelectorAll`

Busca en la página **todos** los elementos que cumplen un selector y te los da en una lista (una `NodeList`) [1].

```
const numeros =
  document.querySelectorAll('[data-numero]');
```

`forEach`

Recorre esa lista y hace lo mismo con cada elemento, uno detrás de otro[4].

```
numeros.forEach(boton => {
  // aquí, "boton" es cada uno
});
```

`addEventListener`

Le dice a un elemento: «cuando te hagan clic, ejecuta esta función»[5].

```
boton.addEventListener('click', () => {
  // esto pasa al hacer clic
});
```



Para entenderlo: el `addEventListener` es como poner un timbre en una puerta. Nadie hace nada hasta que alguien lo toca. Y cuando lo toca, suena exactamente lo que vosotros hayáis decidido.



0

SOLUCIÓN 0

La solución de clase

Agrupamos los botones en tres montones (números, operadores y acciones) y sacamos las cuentas a una función aparte que se llama `calcular()`. Una función que recibe datos, devuelve un resultado y no toca nada más.

SOLUCIÓN 0 · PASO 1



La función `calcular()` : una máquina de zumo

```
function calcular(numero1, numero2, operacion) {
    let resultado = 0;

    switch (operacion) {
        case '+':
            resultado = numero1 + numero2;
            break;
        case '-':
            resultado = numero1 - numero2;
            break;
        case '*':
            resultado = numero1 * numero2;
            break;
        case '/':
            if (numero2 === 0) {
                resultado = 'Error';
            } else {
                resultado = numero1 / numero2;
            }
            break;
    }

    return resultado;
}
```



Para entenderlo: una exprimidora. Le metéis dos naranjas y le decís «zumo». Os devuelve zumo. No le importa de qué frutero vienen las naranjas ni dónde vais a poner el vaso.

Eso es lo que llamamos una **función pura**: todo lo que necesita le entra por los paréntesis y todo lo que produce sale por el `return`. No lee la pantalla. No cambia variables de fuera.

El `switch` mira el valor de `operacion` y salta al `case` que coincide[6]. El `break` es el freno: sin él, seguiría ejecutando los casos de debajo.

¿Y si dividimos entre cero? Devolvemos el texto `'Error'`. Hito 7 resuelto… en parte.



Probadlo: abrid la consola (F12) y escribid `calcular(7, 8, '+')`. Debe salir `15`. Podéis probarla sin tocar un solo botón: esa es la gracia de una función pura.

SOLUCIÓN 0 · PASO 2



La memoria y los tres montones de botones

```
// Memoria de la calculadora
let empezarNumero = false;
let operadorPendiente = null;
let numeroAnterior = null;
let numeroActual = null;

// Los tres montones de botones
const numeros    = document.querySelectorAll('[data-numero]');
const operadores = document.querySelectorAll('[data-operador]');
const acciones   = document.querySelectorAll('[data-accion]');
const pantalla   = document.querySelector('output');
```

`null` significa «aquí todavía no hay nada». No es cero: es vacío a propósito.

Los botones se separan en tres grupos según su etiqueta `data-`. Así, cada grupo tiene su propia forma de reaccionar.



Para entenderlo: es como ordenar la ropa sucia en tres cestos: blanca, color y delicada. Cada cesto va a su programa de lavadora.

10 números + 4 operadores + 2 acciones = 16 botones, 16 escuchadores.

SOLUCIÓN 0 · PASO 3 · HITO 1



Cuando pulsáis un número

```
numeros.forEach(numero => {
    numero.addEventListener('click', () => {
        const valor = numero.dataset.numero;

        if (pantalla.textContent === '0' || empezarNumero) {
            pantalla.textContent = valor;   // sustituye
            empezarNumero = false;
        } else {
            pantalla.textContent += valor;  // pega detrás
        }
    });
});
```

Solo hay dos posibilidades, y el `if` las separa:

1. **Sustituir** lo que hay. Pasa si en pantalla hay un `'0'` (así evitamos el feo `07`) o si venimos de pulsar un operador.
2. **Pegar detrás**. En cualquier otro caso: teclear `1` y luego `2` da `12`.

El `||` se lee «o». Basta con que se cumpla una de las dos condiciones.



Ojo: `textContent` siempre es texto. Por eso `'1' + '2'` da `'12'` y no `3`. Aquí nos viene de maravilla.

SOLUCIÓN 0 · PASO 4 · HITOS 2 Y 5



Cuando pulsáis un operador

```
operadores.forEach(operador => {
    operador.addEventListener('click', () => {
        const valor = operador.dataset.operador;

        if (operadorPendiente !== null && empezarNumero !== true) {
            // Ya había una cuenta a medias: subtotal
            numeroActual = Number(pantalla.textContent);
            numeroAnterior = calcular(numeroAnterior, numeroActual, operadorPendiente);
            pantalla.textContent = numeroAnterior;
        } else {
            // Primera vez: solo guardamos
            numeroAnterior = Number(pantalla.textContent);
        }

        operadorPendiente = (numeroAnterior === 'Error') ? null : valor;
        empezarNumero = true;
    });
});
```

Aquí está el truco de las operaciones encadenadas. Al pulsar un operador nos preguntamos: **¿ya había una cuenta a medias?**

- **No** → guardamos lo de la pantalla en `numeroAnterior` y punto.
- **Sí** → calculamos el subtotal ya, lo enseñamos y lo guardamos como nuevo «número de antes».

`Number()` convierte el texto de la pantalla en un número de verdad[7]. Sin eso, `calcular` sumaría textos.

Y al final levantamos la bandera `empezarNumero = true`: el próximo dígito empieza número nuevo.

SOLUCIÓN 0 · TRAZA EN LA PIZARRA



Seguimos a mano: `5 + 3 + 2 =`

PULSO	PANTALLA	NUMEROANTERIOR	OPERADORPENDIENTE	EMPEZARNUMERO	¿QUÉ HA PASADO?
`5`	5	null	null	false	Había un 0: se sustituye.
`+`	5	5	'+'	true	No había cuenta a medias: solo se guarda.
`3`	3	5	'+'	false	Bandera arriba: se sustituye el 5.
`+`	**8**	8	'+'	true	**¡Subtotal!** 5 + 3 = 8.
`2`	2	8	'+'	false	Bandera arriba: se sustituye el 8.
`=`	**10**	10	null	true	8 + 2 = 10. Operación cerrada.

Hacedla vosotros en papel antes de mirar la solución. Si os sale igual, habéis entendido la Solución 0. Así de simple.

SOLUCIÓN 0 · PASO 5 · HITOS 3 Y 6



El igual y el borrar comparten montón

```
acciones.forEach(accion => {
    accion.addEventListener('click', () => {
        const valor = accion.dataset.accion;

        switch (valor) {
            case 'igual':
                if (operadorPendiente !== null) {
                    numeroActual = Number(pantalla.textContent);
                    numeroAnterior = calcular(numeroAnterior, numeroActual, operadorPendiente);
                    pantalla.textContent = numeroAnterior;
                }
                operadorPendiente = null;
                empezarNumero = true;
                break;

            case 'borrar':
                pantalla.textContent = '0';
                operadorPendiente = null;
                empezarNumero = true;
                break;
        }
    });
});
```

Los dos botones de acción viven en el mismo grupo, así que un `switch` decide qué hace cada uno.

Igual: si hay una cuenta pendiente, la resuelve con `calcular()`, la pinta y cierra la operación (`operadorPendiente = null`).

Borrar: pone la pantalla a `'0'` y olvida la operación.



Ojo: fijaos en que el bloque del igual repite casi letra por letra el subtotal del operador. Cuando veáis código repetido, pensad: «esto podría ser una función». Esa idea la recoge la Solución 1.

SOLUCIÓN 0 · BALANCE



Lo bueno y lo mejorable

Lo que hace bien

- La función `calcular()` es pura: se prueba sola en la consola.
- Agrupar por tipo de botón es muy natural de leer.
- Si una cuenta da `Error`, anula el operador y no sigue operando con él.

Lo que se puede mejorar

- **No redondea.** `10 / 3 =` muestra `3.3333333333333335`. El hito 7 queda a medias.
- El cálculo del igual y del subtotal está duplicado.
- La pantalla hace de variable: mezcla lo que se ve con lo que se guarda.
- 16 escuchadores, uno por botón.



Probadlo: pulsad `1 0 / 3 =` en la versión 0. Luego haced lo mismo en la versión 1. Comparad lo que sale.



1

SOLUCIÓN 1

Un escuchador por botón

Cada botón sigue teniendo su propio `addEventListener`, pero ya no piensa: solo pasa el recado a una función con nombre propio. `agregarNumero`, `seleccionarOperador`, `calcular` y `borrar`.

SOLUCIÓN 1 · LA IDEA



El botón avisa, la función trabaja



Para entenderlo: en un restaurante, el camarero no cocina. Toma nota y se la pasa a la cocina. Si mañana cambiáis la receta, no tenéis que reeducar a los camareros: solo al cocinero.

En la Solución 0, el código de «qué hacer» estaba **dentro** de cada escuchador. Aquí lo sacamos fuera, a funciones con un nombre que se entiende al leerlo.

Y aparece una función nueva muy humilde: `actualizarDisplay()`. Es la **única** que toca la pantalla.

```
// 1. Selección del DOM: 5 selectores
const display = document.getElementById('display');
const botonesNumeros    = document.querySelectorAll('[data-numero]');
const botonesOperadores = document.querySelectorAll('[data-operador]');
const botonIgual  = document.querySelector('[data-accion="igual"]');
const botonBorrar = document.querySelector('[data-accion="borrar"]');

// 2. Memoria
let operandoActual = '0';
let operandoAnterior = null;
let operacion = null;
let reiniciarDisplay = false;

// 3. La única puerta a la pantalla
function actualizarDisplay() {
    display.textContent = operandoActual;
}
```



Ojo: `operandoActual` es **texto** (`'0'`, con comillas). Lo guardamos así a propósito para poder pegar dígitos: `'1' + '2'` → `'12'`. Solo lo convertiremos en número en el momento de calcular.

SOLUCIÓN 1 · HITOS 1 Y 2



`agregarNumero` y `seleccionarOperador`

```
function agregarNumero(numero) {
    if (reiniciarDisplay) {
        operandoActual = numero;
        reiniciarDisplay = false;
    } else {
        operandoActual = (operandoActual === '0')
            ? numero
            : operandoActual + numero;
    }
    actualizarDisplay();
}

function seleccionarOperador(nuevoOperador) {
    if (operacion !== null && !reiniciarDisplay) {
        calcular();          // subtotal (hito 5)
    }
    operandoAnterior = operandoActual;
    operacion = nuevoOperador;
    reiniciarDisplay = true;
}
```

La lógica es la misma que en la Solución 0, pero ahora trabajamos sobre la variable y al final llamamos a `actualizarDisplay()`.

Esa línea con `?` y `:` es el **operador ternario**[8]. Es un `if/else` de bolsillo:

```
condición ? siEsVerdad : siEsMentira
```

Leedlo así: «¿hay un 0? Entonces pon el número; si no, pégalo detrás».

En `seleccionarOperador`, si ya había una cuenta a medias, llamamos a `calcular()` antes de apuntar la nueva operación. Mismo truco del subtotal, sin copiar código.

SOLUCIÓN 1 · HITOS 3, 4 Y 7



`calcular()` : ahora sin parámetros

```
function calcular() {
    if (operacion === null || operandoAnterior === null) return;

    const anterior = parseFloat(operandoAnterior);
    const actual   = parseFloat(operandoActual);
    let resultado;

    switch (operacion) {
        case '+': resultado = anterior + actual; break;
        case '−': resultado = anterior − actual; break;
        case '*': resultado = anterior * actual; break;
        case '/': resultado = (actual === 0) ? 'Error' : anterior / actual; break;
        default: return;
    }

    if (typeof resultado === 'number') {
        resultado = Math.round(resultado * 100000000) / 100000000;
    }

    operandoActual = resultado.toString();
    operacion = null;
    operandoAnterior = null;
    reiniciarDisplay = true;
    actualizarDisplay();
}
```

Primera línea, el portero: si no hay nada que calcular, `return` y fuera. Se llama «cláusula de guarda» y os ahorra un `if` gigante.

`parseFloat()` convierte el texto en número[9]. Hace lo mismo que `Number()` en nuestro caso.

El redondeo multiplica por cien millones, redondea al entero con `Math.round`[10] y vuelve a dividir. Resultado: como mucho 8 decimales. Los números de JavaScript tienen precisión limitada[7], y por eso `10 / 3` sin redondear da ese `…35` final tan raro.



Ojo: esta `calcular()` **no** es pura: lee y modifica las variables de fuera. Es cómoda, pero ya no podéis probarla sola en la consola como en la Solución 0.

SOLUCIÓN 1 · HITO 6 Y LOS ESCUCHADORES



`borrar()` y quién avisa a quién

```
function borrar() {
    operandoActual = '0';
    operandoAnterior = null;
    operacion = null;
    reiniciarDisplay = false;
    actualizarDisplay();
}

// 5. Escuchadores: solo pasan el recado
botonesNumeros.forEach((boton) => {
    boton.addEventListener('click', () => {
        agregarNumero(boton.dataset.numero);
    });
});

botonesOperadores.forEach((boton) => {
    boton.addEventListener('click', () => {
        seleccionarOperador(boton.dataset.operador);
    });
});

botonIgual.addEventListener('click', () => calcular());
botonBorrar.addEventListener('click', () => borrar());
```

`borrar()` deja las cuatro variables exactamente como estaban al cargar la página. «Estado de fábrica».

Mirad ahora los escuchadores. Cada uno tiene **una sola línea** dentro: llamar a la función que toca con el dato del botón.

Si mañana queréis que la calculadora funcione también con el teclado del ordenador, ya tenéis las funciones hechas. Solo tendríais que llamarlas desde otro sitio.

Cuenta de escuchadores: 10 + 4 + 1 + 1 = **16**. Los mismos que en la Solución 0.

SOLUCIÓN 1 · TRAZA EN LA PIZARRA



Seguimos a mano: `8 / 0 =`

PULSO	OPERANDOACTUAL	OPERANDOANTERIOR	OPERACION	REINICIARDISPLAY	¿QUÉ HA PASADO?
`8`	'8'	null	null	false	Había '0': se sustituye.
`/`	'8'	'8'	'/'	true	Se apunta el 8 y la división.
`0`	'0'	'8'	'/'	false	Bandera arriba: se sustituye.
`=`	**'Error'**	null	null	true	`actual === 0` → 'Error'.

Lo bueno

- Funciones con nombre: se lee casi como un texto.
- Una sola función toca la pantalla.
- Redondeo incluido: hito 7 completo.

Lo mejorable

- Sigue habiendo 16 escuchadores.
- 5 selectores distintos para un mismo teclado.
- `calcular()` depende de variables globales.



2

SOLUCIÓN 2

Delegación de eventos

Las funciones son idénticas a las de la Solución 1. Lo que cambia es cómo nos enteramos de los clics: en lugar de 16 escuchadores, ponemos **uno solo** en el contenedor `.teclado`.

SOLUCIÓN 2 · EL CONCEPTO



El clic sube como una burbuja

Cuando hacéis clic en un botón, el navegador no avisa solo al botón. Avisa al botón, luego a su padre, luego al padre de su padre… hasta llegar arriba del todo. Eso se llama **burbujeo** (*event bubbling*)[11].



Para entenderlo: el conserje de un edificio. En lugar de poner un timbre distinto en cada piso con su propio vigilante, todos los paquetes llegan a la portería. El conserje mira la etiqueta del paquete y lo sube al piso que toca.

Así que, si ponemos el escuchador en `.teclado`, se entera de **todos** los clics de sus botones. Uno para gobernarlos a todos.

```
<body>
```

↑ sube

```
<main class="calculadora">
```

↑ sube

```
<div class="teclado"> ← aquí escuchamos
```

↑ sube

```
<button data-numero="7"> ← aquí hacéis clic
```

SOLUCIÓN 2 · EL CÓDIGO



Un único escuchador hace de conserje

```
const display = document.getElementById('display');
const teclado = document.querySelector('.teclado');

teclado.addEventListener('click', (evento) => {
    // 1. ¿Qué botón se ha pulsado?
    const boton = evento.target.closest('button');
    if (!boton) return;

    // 2. Leemos su etiqueta
    const numero   = boton.dataset.numero;
    const operador = boton.dataset.operador;
    const accion   = boton.dataset.accion;

    // 3. Repartimos el trabajo
    if (numero !== undefined) {
        agregarNumero(numero);
    } else if (operador !== undefined) {
        seleccionarOperador(operador);
    } else if (accion === 'igual') {
        calcular();
    } else if (accion === 'borrar') {
        borrar();
    }
});
```

Paso 1. `evento.target` es el elemento exacto donde cayó el clic[12]. `closest('button')` busca, desde ahí hacia arriba, el botón más cercano[13]. Si el clic cayó en el hueco entre dos botones, no hay botón: `null` → `return`.

Paso 2. Leemos las tres etiquetas posibles. Un botón solo tiene una; las otras dos valdrán `undefined`[2].

Paso 3. Según cuál tenga valor, llamamos a la función que toca. El conserje sube el paquete al piso correcto.

SOLUCIÓN 2 · LO QUE PIENSA EL CONSERJE



Seguimos a mano un clic en la tecla 7

En clase lo veis en directo con la calculadora de la presentación. Aquí os lo dejo paso a paso, tal y como lo ejecuta el navegador cuando pulsáis el `7` después de haber tecleado `5 +`:

PASO	QUÉ PASA	VALOR
1	Elemento exacto donde cae el clic: `evento.target`	`<button data-numero="7">`
2	El clic sube hasta `.teclado`, que es el único que escucha	Se ejecuta su función
3	`evento.target.closest('button')`	El mismo botón del 7
4	Lectura de etiquetas: `numero` · `operador` · `accion`	`'7'` · `undefined` · `undefined`
5	Como `numero` ≠ `undefined`, se llama a…	`agregarNumero('7')`
6	Memoria al terminar	actual `'7'` · anterior `'5'` · operación `'+'` · reiniciar `false`



Probadlo: si el clic cae en el hueco entre dos teclas, `evento.target` es el propio `.teclado`, `closest('button')` devuelve `null` y el `return` corta: no pasa nada.

SOLUCIÓN 2 · DOS DETALLES FINOS



¿Por qué `!== undefined` y no un simple `if (numero)` ?

Si un botón no tiene `data-numero`, `boton.dataset.numero` vale `undefined`. Por eso preguntamos exactamente eso.

«Ya, Ramón, pero `if (numero)` es más corto». Cierto. Y en este caso funcionaría, porque `'0'` como **texto** cuenta como verdadero (es *truthy*)[14].



Ojo: pero si algún día ese valor fuera el **número** `0` o un texto vacío `''`, el `if` lo trataría como falso y la tecla dejaría de funcionar. Comparar con `undefined` dice exactamente lo que queréis decir. Sin sustos.

Botones nuevos, cero cambios

Con 16 escuchadores, un botón que se añade después de cargar la página no tiene escuchador: no funciona hasta que le pongáis uno.

Con delegación, el escuchador está en el padre. Cualquier botón nuevo con su `data-numero` funciona al momento. En la guía paso a paso tenéis una prueba para comprobarlo con una tecla «00».

Escuchadores: **1**. Selectores: **1** (más el display).



Probadlo: en la consola del navegador: `Boolean('0')` → `true` ; `Boolean(0)` → `false`. Ahí suele estar el lío.

COMPARATIVA · FRENTE A FRENTE



Las tres soluciones en una tabla

CRITERIO	0 · CLASE	1 · INDIVIDUALES	2 · DELEGACIÓN
Cómo agrupa los botones	3 listas por tipo	5 selectores	1 contenedor `.teclado`
Escuchadores activos	16	16	**1**
Dónde vive la lógica	Dentro de cada escuchador	Funciones con nombre	Funciones con nombre
Función de cálculo	**Pura**: `calcular(n1, n2, op)`	Usa variables globales	Usa variables globales
Número en pantalla	Se lee de la propia pantalla	Variable `operandoActual`	Variable `operandoActual`
Redondeo de decimales	No	8 decimales	8 decimales
Botones añadidos después	Necesitan escuchador nuevo	Necesitan escuchador nuevo	Funcionan solos
Concepto nuevo que enseña	Función pura y `switch`	Separar «quién avisa» de «qué se hace»	Burbujeo, `closest`, `target`

COMPARATIVA · EL MISMO HITO, TRES CÓDIGOS



¿Cómo se entera cada una de que pulsáis un número?

Solución 0

```
numeros.forEach(numero ⇒ {
  numero.addEventListener('click', () ⇒ {
    const valor = numero.dataset.numero;
    // ... toda la lógica aquí dentro
  });
});
```

El escuchador piensa y trabaja.

Solución 1

```
botonesNumeros.forEach(boton ⇒ {
  boton.addEventListener('click', () ⇒ {
    agregarNumero(boton.dataset.numero);
  });
});
```

El escuchador solo pasa el recado.

Solución 2

```
teclado.addEventListener('click', e ⇒ {
  const boton = e.target.closest('button');
  if (!boton) return;
  if (boton.dataset.numero ≠= undefined)
    agregarNumero(boton.dataset.numero);
});
```

Un solo escuchador para todos.

Fijaos en la evolución: de «cada botón lo hace todo», a «cada botón avisa», a «un único conserje para todo el teclado».

COMPARATIVA · TRAMPAS TÍPICAS



Los fallos que más veo en esta práctica

SÍNTOMA EN PANTALLA	CAUSA	ARREGLO
`07`, `007`…	No se comprueba si hay un `'0'` antes de pegar.	Si la pantalla es `'0'`, sustituir en vez de concatenar.
`15 + 4` → se ve `154`	Falta la bandera de «número nuevo».	Activarla al pulsar operador y bajarla con el primer dígito.
`7 + 8 =` → `78`	Se suman textos, no números.	`Number()` o `parseFloat()` antes de operar[7][9].
`9 + 4 =` → `5`	Falta un `break` en el `switch`.	Un `break` al final de cada `case`[6].
`10 / 3 =` → `3.3333333333333335`	Precisión limitada de los números en JS.	Redondear con `Math.round`[10].
Tras `Error`, `+ 3 =` → `NaN`	Se sigue operando con el texto `'Error'`. Pasa en las tres soluciones.	Reto para vosotros: lo tenéis en los retos para casa.

COMPARATIVA · MI RECOMENDACIÓN



¿Cuál uso?

Solución 0

Para **aprender**. Todo está a la vista dentro de cada escuchador y la función `calcular()` pura se prueba sola.

Solución 1

Para **ordenar**. Cuando el código empieza a crecer, las funciones con nombre os salvan la vida.

Solución 2

Para **trabajar de verdad**. Muchos elementos iguales o creados sobre la marcha: un escuchador en el padre.

Retos para casa

1. Arreglad el `NaN` que aparece al operar después de un `Error`.
2. Añadid el redondeo a la Solución 0 sin romper su función pura.
3. Llevad a la Solución 2 la idea de la 0: que `calcular(anterior, actual, operacion)` reciba los datos y devuelva el resultado.
4. Añadid una tecla `.` para decimales. Pista: no dejéis que se escriban dos puntos.

COMPROBAD LO APRENDIDO



Cinco preguntas rápidas

1. ¿Qué cuatro cosas tiene que recordar la calculadora?
2. ¿Para qué sirve la bandera `empezarNumero` / `reiniciarDisplay`?
3. ¿Por qué la `calcular()` de la Solución 0 se puede probar sola en la consola y la de la 1 no?
4. ¿Qué es el burbujeo y por qué permite usar un solo escuchador?
5. ¿Qué devuelve `closest('button')` si hacéis clic entre dos teclas?

Probad las tres versiones abriendo el `index.html` de cada carpeta: `version-0-solucion-clase`, `version-1-escuchadores-individuales` y `version-2-delegacion-eventos`.



Si habéis llegado hasta aquí entendiendo las trazas, ya sabéis más de eventos que mucha gente que los usa a diario.



Fuentes y Documentación Oficial

1. MDN Web Docs — Document.querySelectorAll()
2. MDN Web Docs — HTMLElement: dataset property
3. MDN Web Docs — Node.textContent
4. MDN Web Docs — NodeList.prototype.forEach()
5. MDN Web Docs — element.addEventListener
6. MDN Web Docs — switch
7. MDN Web Docs — Number
8. MDN Web Docs — Operador condicional (ternario)
9. MDN Web Docs — parseFloat()
10. MDN Web Docs — Math.round()
11. MDN Web Docs — Burbujeo de eventos
12. MDN Web Docs — Event.target
13. MDN Web Docs — Element.closest()
14. MDN Web Docs — Truthy